RTK Session Benchmark

Public

Task:	**Multi-task: GitHub OpenCode - Git & GH CLI, JS Moleculer - NPM & ESLint & TSC & Jest, T**
Model:	**claude-opus-4-7**
VM pairs:	**10**
Date:	**2026-06-12 12:58**
RTK Version:	**latest**
Max Turns:	**25**
Timeout:	**15 min**
Codebase:	**https://github.com/anomalyco/opencode @ dev | https://github.com/moleculerjs/moleculer**

Executive Summary

Bash output (RTK target): +42.3% savings (significant) across 50 VM pair(s). Cost: +19.0% savings.

Causal Metric Chain

RTK filters bash output -> Claude behavior may change (more/fewer turns) -> total input changes -> cost changes

1. DIRECT: Bash Result Bytes *RTK filters this output. The one thing RTK directly controls.*	ON: 24,186 | OFF: 41,886 |	**+42.3%**
2. BEHAVIORAL: API Calls (Turns) *Did filtering change Claude's strategy? More turns = RTK may be too aggressive.*	ON: 15 | OFF: 15 |	**-0.1%**
3. COMBINED: Total Input Tokens *Net result of filtering + turn count. Each turn replays full conversation.*	ON: 500,240 | OFF: 557,771 |	**+10.3%**
4. BOTTOM LINE: Cost USD *Final cost impact. Driven by input tokens, caching, and output tokens.*	ON: $0.3973 | OFF: $0.4903 |	**+19.0%**

Context: Bash = 100% of tool output, ~1.9% of total input. API turns identical: No

Verdicts

Bash Result Bytes:	RTK ON (+42.3%) *
Total Input Tokens:	RTK ON (+10.3%) *
Cache Read Tokens:	RTK ON (+8.9%) *
Cache Creation Tokens:	RTK ON (+42.3%) *

Output Tokens:	RTK ON (+9.6%) *
Cost USD:	RTK ON (+19.0%) *
API Calls:	RTK OFF (-0.1%)
Total Duration Ms:	RTK ON (+2.3%)
All Tool Result Bytes:	RTK ON (+41.9%) *

** p < 0.05 -- statistically significant (less than 5% chance the difference is due to random variance)*

Benchmark Configuration

Task

Name: Multi-task: GitHub OpenCode - Git & GH CLI, JS Moleculer - NPM & ESLint & TSC & Jest, TS SWR - TypeS

Description: GitHub workflow audit using git branch/status/stash and gh pr/issue/run/repo commands (read-only) | Code q

Codebase: https://github.com/anomalyco/opencode @ dev | https://github.com/moleculerjs/moleculer @ 9441807 | https

Prompt:

```
[GitHub OpenCode - Git & GH CLI] This is a TypeScript open source coding agent project called
opencode. Produce a | [JS Moleculer - NPM & ESLint & TSC & Jest] This is a Node.js
microservices framework called Moleculer. Produce a code quali | [TS SWR - TypeScript &
Vitest] This is SWR, a TypeScript data-fetching library by Vercel. Produce a code qualit |
[Python Pretix - Ruff & Mypy & Pip] This is a Django-based ticket sales platform called
pretix. Produce a code quali | [Rust Abtop - Cargo Build ...
```

Execution Parameters

Model: claude-opus-4-7

Max Turns: 25

Timeout: 15 min

VM Pairs: 10 (total VMs: 20)

RTK Version: latest

Claude Code Configuration

Flags:

```
--dangerously-skip-permissions
--output-format stream-json --verbose
--model claude-opus-4-7
--exclude-dynamic-system-prompt-sections
--max-turns 25
--append-system-prompt <...>
```

System Prompt:

```
[GitHub OpenCode - Git & GH CLI] MANDATORY: Execute all 12 commands using the Bash tool, in
order, one per call.
 | [JS Moleculer - NPM & ESLint & TSC & Jest] MANDATORY: Execute all 12 commands using the
Bash tool, in order, one per call.
 | [TS SWR - TypeScript & Vitest] MANDATORY: Execute all 14 commands using the Bash tool, in
order, one per call.
 | [Python Pretix - Ruff & Mypy & Pip] MANDATORY: Execute all 12 commands using the Bash tool,
in order, one per call.
 | [Rust Abtop - Cargo Build ...
```

RTK Setup

Hook: PreToolUse hook on Bash matcher

CLAUDE.md: RTK instructions injected (prefix commands with rtk)

Environment

Container: Ubuntu 24.04 (Docker)

Toolchain: Rust (rustup), Claude Code CLI

Metrics Glossary

Metric	Unit	What It Measures	Why It Matters
Bash Result Bytes	bytes	**Total bytes of bash command output (cargo test, cargo clippy, etc.) returned to Claude as tool results.**	**PRIMARY KPI. RTK filters this output. Fewer bytes = fewer input tokens = lower cost.**
Total Input Tokens	tokens	TRUE total input: uncached + cache_read + cache_creation. Everything sent to the API per session.	The real total input. Use this (not 'Input Tokens') to understand actual data volume sent to the API.
Input Tokens (Uncached)	tokens	Tokens that were NOT served from cache and NOT newly written to cache. Typically very small.	Often near zero due to aggressive prompt caching. Misleading if read as 'total input'.
Cache Read Tokens	tokens	Tokens served from Anthropic's prompt cache instead of being re-processed. Usually the largest component.	Cached tokens cost ~90% less. Dominates total input in most sessions.
Cache Creation Tokens	tokens	Tokens newly written to the prompt cache on this request.	One-time cost per new cache entry. Priced ~25% above uncached tokens.
Output Tokens	tokens	Total tokens generated by Claude (responses + tool calls).	Secondary cost factor. Less affected by RTK since RTK filters input, not output.
Cost USD	USD	Total API cost for the session (all models combined).	Bottom-line cost impact. Driven by input tokens, caching, and output tokens.
API Calls	calls	Number of agentic turns (API round-trips). Each turn replays the FULL conversation to the API.	BEHAVIORAL metric. If ON != OFF, RTK changed Claude's strategy. More turns from aggressive filtering can negate byte savings.
Total Duration	ms	Wall-clock time for the entire session.	End-to-end time including API latency and command execution.
All Tool Result Bytes	bytes	Total bytes from ALL tool results (Bash + Read + Edit + Grep + etc.).	Shows what fraction of tool output is Bash vs other tools. Context for bash_result_bytes.

Methodology

Experimental Design

Each benchmark run deploys N identical VM pairs. Within each pair, one VM runs with RTK ON (PreToolUse hook active, compressing Bash output) and one with RTK OFF (baseline, no hook). Both VMs receive the same task prompt, system prompt, model, and Claude Code flags. ON VMs additionally receive a CLAUDE.md file with RTK instructions (e.g. 'prefix commands with rtk') and the RTK binary -- this extra context is inherent to RTK's design. VMs are paired by their semantic index extracted from the VM name (e.g. rtk-bench-on-3 pairs with rtk-bench-off-3).

Sample size: n_on=50, n_off=50 (after filtering).

Data Collection

Two metric sources are tried in order:
1. OTEL log (if present): per-API-request events provide individual token counts, cost, and duration.
2. Fallback: stream-json stdout -- the 'result' event at session end provides aggregate token counts and cost.

Extracted from the result event (or summed from OTEL events):
- input_tokens (uncached), cache_read_input_tokens, cache_creation_input_tokens, output_tokens
- total_cost_usd: actual cost reported by the Claude API (not computed from token prices)
- num_turns: number of agentic turns (API round-trips)
- duration: from OTEL, total_duration_ms is the sum of per-request duration_ms; from stream-json, it is duration_api_ms from the result event

api_calls equals num_turns from the result event (stream-json mode) or the count of api_request OTEL events (OTEL mode). In stream-json mode, placeholder events are added so api_call_count matches num_turns.

Bash result bytes are measured per tool invocation: UTF-8 byte length of each tool_result content block in the stream-json user events, matched to tool_use blocks by tool_use_id. Only results from the Bash tool are counted for bash_result_bytes; all_tool_result_bytes includes all tools.

Session Filtering

Sessions are excluded from the statistical comparison if:
1. The session has a recorded error (API failure, timeout, etc.)
2. No API request events were parsed (empty api_requests list)
3. The parsed metrics have zero uncached input tokens (input_tokens = 0, indicates no API calls completed, e.g. auth failure or credit exhaustion)

Conditions 2 and 3 are checked together: a session must have at least one API request AND non-zero uncached input tokens to be included.

This may result in unequal ON/OFF sample sizes. The statistical test adapts accordingly (see below).

Statistical Tests

Two analyses are performed:

1. Group-level comparison (per metric):
Compares the mean of all ON sessions vs all OFF sessions.
- If n_on == n_off: paired t-test (scipy.stats.ttest_rel), pairing by list position in manifest order. List positions correspond to VM indices when no sessions are filtered; if asymmetric filtering removes different VMs but leaves equal counts,

pairings may be misaligned.
- If n_on != n_off: independent samples t-test (scipy.stats.ttest_ind) as fallback
- If either group has < 2 samples: p-value defaults to 1.0 (no test)

2. Per-VM pair analysis:
Pairs ON/OFF sessions by semantic index extracted from VM name (e.g. rtk-bench-on-3 -> index 3). Only matching indices are paired; unmatched sessions are excluded. This is robust to asymmetric filtering.

Significance threshold: p < 0.05 (two-tailed). Marked with * in the report. Results marked (i) used the independent t-test fallback.

95% Confidence Intervals (t-distribution):
CI = mean +/- t(0.975, n-1) * (stddev / sqrt(n))

Standard deviation uses Bessel's correction (n-1 denominator).

Metrics Compared

All metrics follow 'lower is better' convention.

Primary KPI:
bash_result_bytes -- total Bash tool output in bytes. This is what RTK directly compresses.

Token diagnostics (group-level comparison):
total_all_input_tokens = cache_read + cache_creation + uncached input. Raw count; does not reflect cost (each type has a different price).
cache_read_tokens -- served from prompt cache (~90% cheaper)
cache_creation_tokens -- written to cache (~25% more expensive)
output_tokens -- tokens generated by the model

Cost & operational:
cost_usd -- actual cost from Claude API (correctly weights all token types by their per-token price)
api_calls -- equals num_turns (see Data Collection for source details)
total_duration_ms -- sum of per-request duration (OTEL) or duration_api_ms from result event (stream-json)

Tool output:
all_tool_result_bytes -- total output from all tools (Bash + Read + Glob + Edit + etc.)

Per-VM pair analysis additionally includes:
input_tokens -- uncached input tokens only (not in group comparison)

Savings Calculation

Per-metric savings (group level):
savings_pct = (1 - ON_mean / OFF_mean) * 100
Positive = RTK ON uses fewer resources (savings)
Negative = RTK ON uses more resources (loss)

Per-VM pair overall savings:
overall = cost_usd savings percentage for that pair
(direct dollar comparison, no weighting formula)

API Calls Interpretation

ON < OFF: RTK helped Claude focus (good filtering)
ON > OFF: RTK may have removed signal, causing extra turns (over-filtering)
ON == OFF: RTK did not change Claude's agentic behavior

Each extra turn replays the full growing context (~25K+ tokens), often negating any per-turn byte savings from RTK compression.

Aggregate Comparison

Lower is better for all metrics. Green = RTK ON wins, Red = RTK OFF wins.

Metric	OFF Mean	OFF SD	ON Mean	ON SD	ON 95% CI	Svgs%	p-val	Winner
Bash Result Bytes	**41,886**	**16,267**	**24,186**	**14,487**	**20,069-28,303**	**+42.3%**	**0.000**	**RTK ON**
Total Input Tokens	557,771	191,269	500,240	207,990	441,130-559,350	+10.3%	0.000	RTK ON
Cache Read Tokens	534,305	182,857	486,690	202,902	429,026-544,354	+8.9%	0.000	RTK ON
Cache Creation Tokens	23,447	11,576	13,531	7,064	11,524-15,539	+42.3%	0.000	RTK ON
Output Tokens	3,026	770	2,736	774	2,516-2,956	+9.6%	0.001	RTK ON
Cost USD	$0.4903	$0.1679	$0.3973	$0.1545	$0.3534-$0.4412	+19.0%	0.000	RTK ON
API Calls (Behavi	**15**	**3**	**15**	**4**	**13-16**	**-0.1%**	**0.939**	**RTK OFF**
Total Duration Ms	65,607	14,924	64,106	15,971	59,568-68,645	+2.3%	0.590	RTK ON
All Tool Result Bytes	41,994	16,368	24,402	14,691	20,227-28,577	+41.9%	0.000	RTK ON

** p < 0.05 -- statistically significant. (i) = independent t-test (unequal ON/OFF sample counts).*

Per-Task Breakdown

Key savings metrics broken down per task. Positive = RTK ON saves vs OFF baseline.

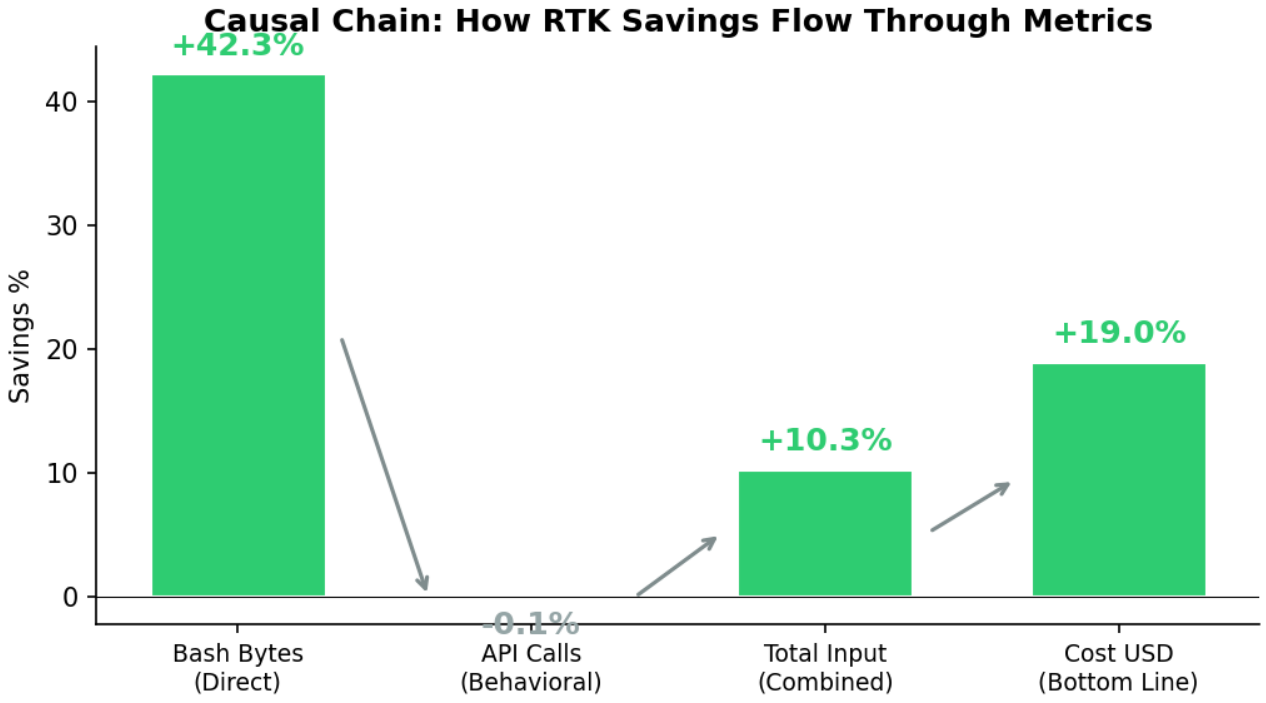
Task	n	Bash Bytes	Total Input	Cost	API Calls
GitHub OpenCode - Git & GH CLI	10/10	+63.7%	+7.4%	+14.3%	+0.0%
JS Moleculer - NPM & ESLint & TSC &	10/10	+38.7%	+18.5%	+21.4%	+0.7%
TS SWR - TypeScript & Vitest	10/10	+36.5%	-2.0%	+12.4%	-13.3%
Python Pretix - Ruff & Mypy & Pip	10/10	+24.6%	+11.2%	+16.6%	+4.4%
Rust Abtop - Cargo Build & Test & C	10/10	+78.1%	+25.3%	+36.6%	+15.3%

n = ON sessions / OFF sessions included after filtering.

Deep Analysis

Causal Chain: How Savings Flow

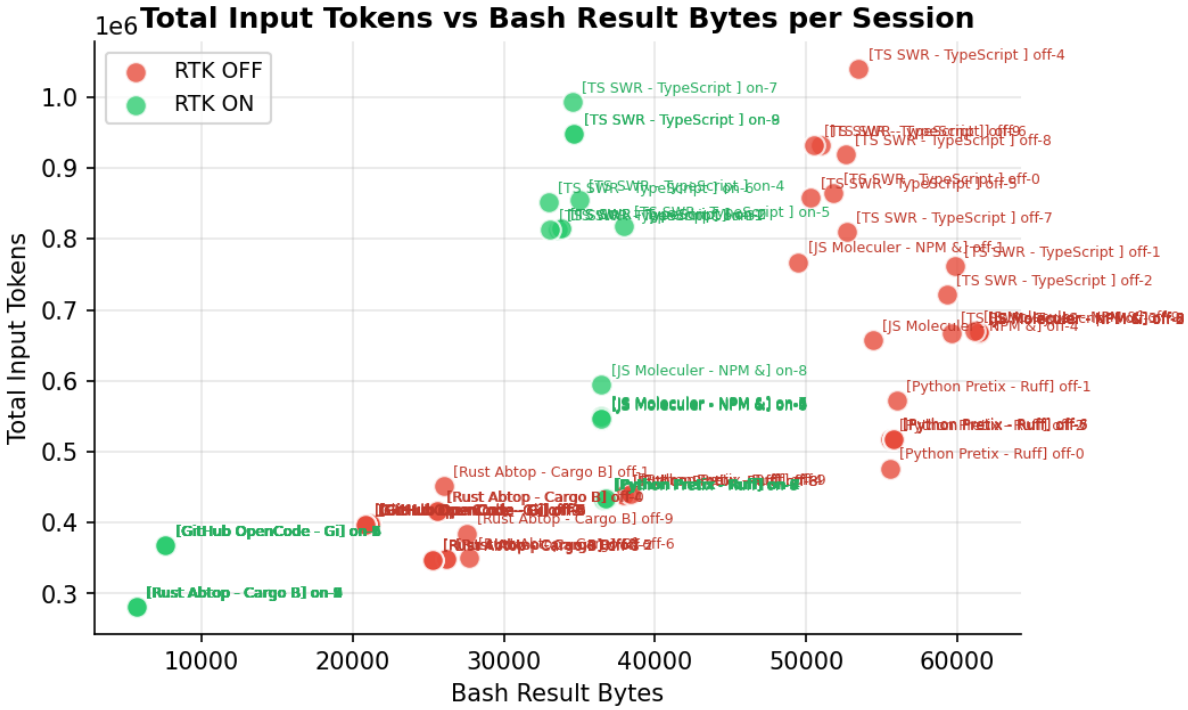
Each metric in the chain is affected by the previous one. Green = RTK ON saves, Red = RTK ON costs more.



RTK filtered 42.3% of bash output, leading to 19.0% cost savings.

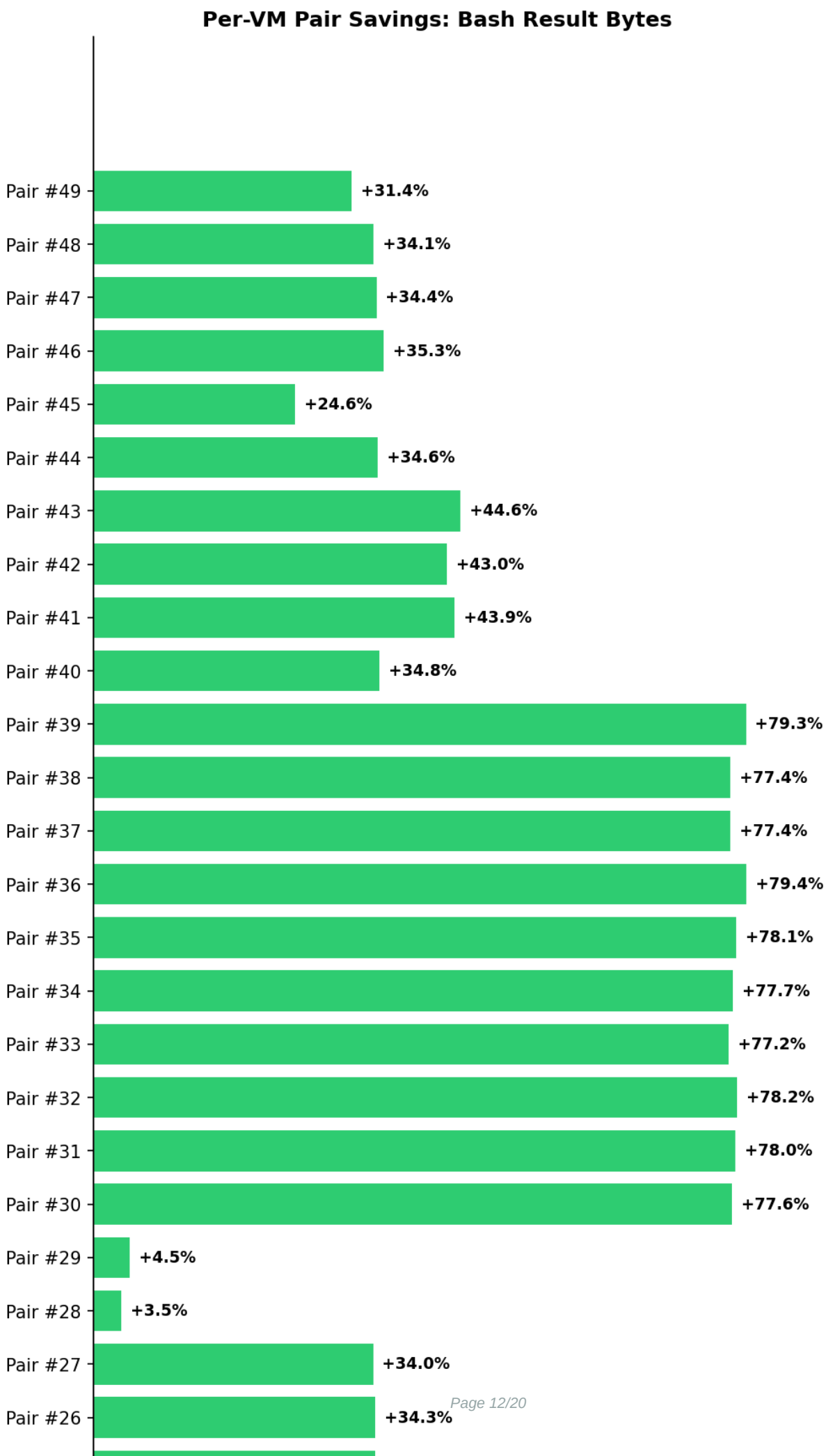
Per-Session Analysis

Each dot is one Claude Code session. Clustering reveals patterns; scattered points suggest non-determinism or behavioral differences.

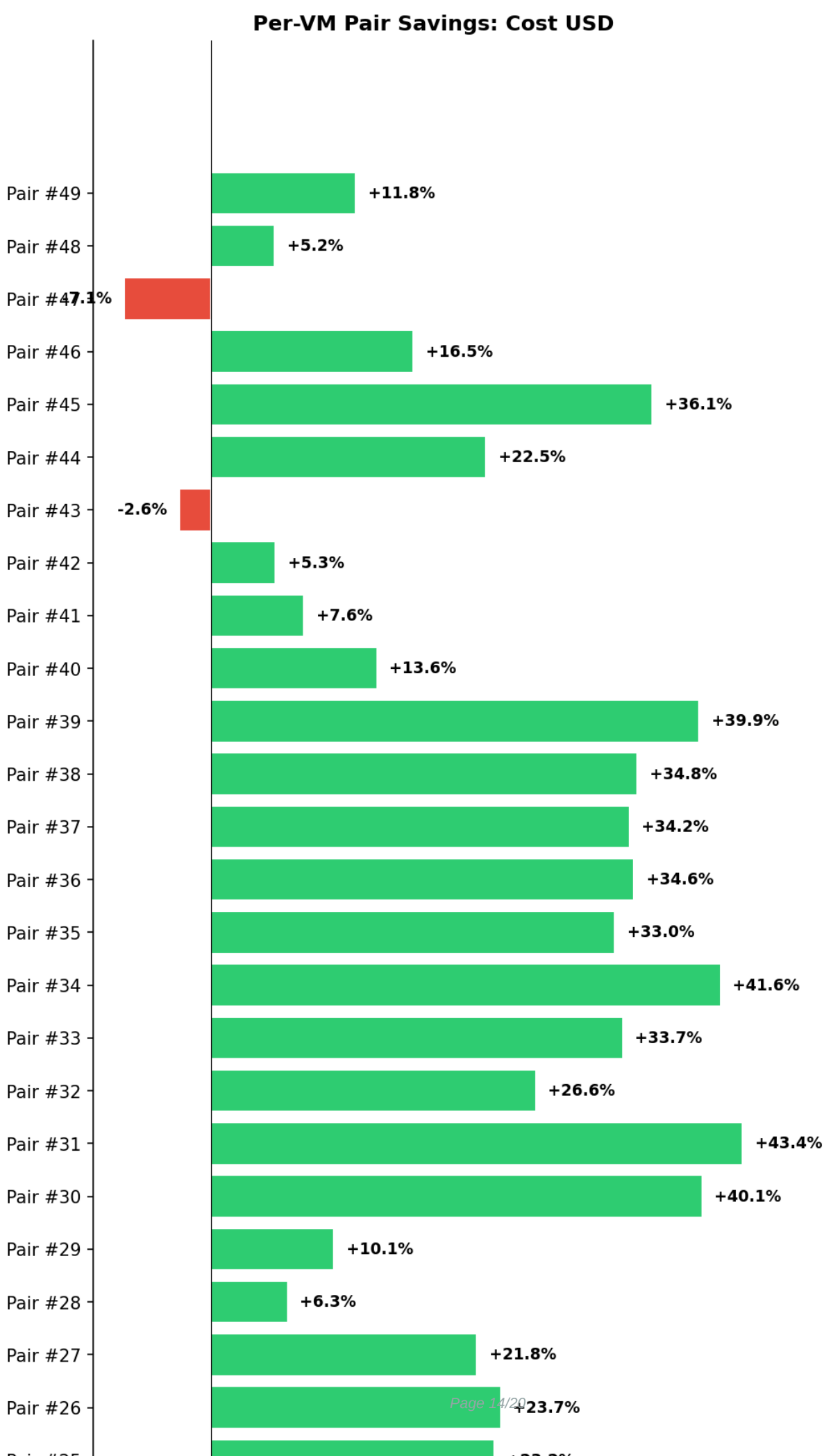


Per-VM Pair Savings Breakdown

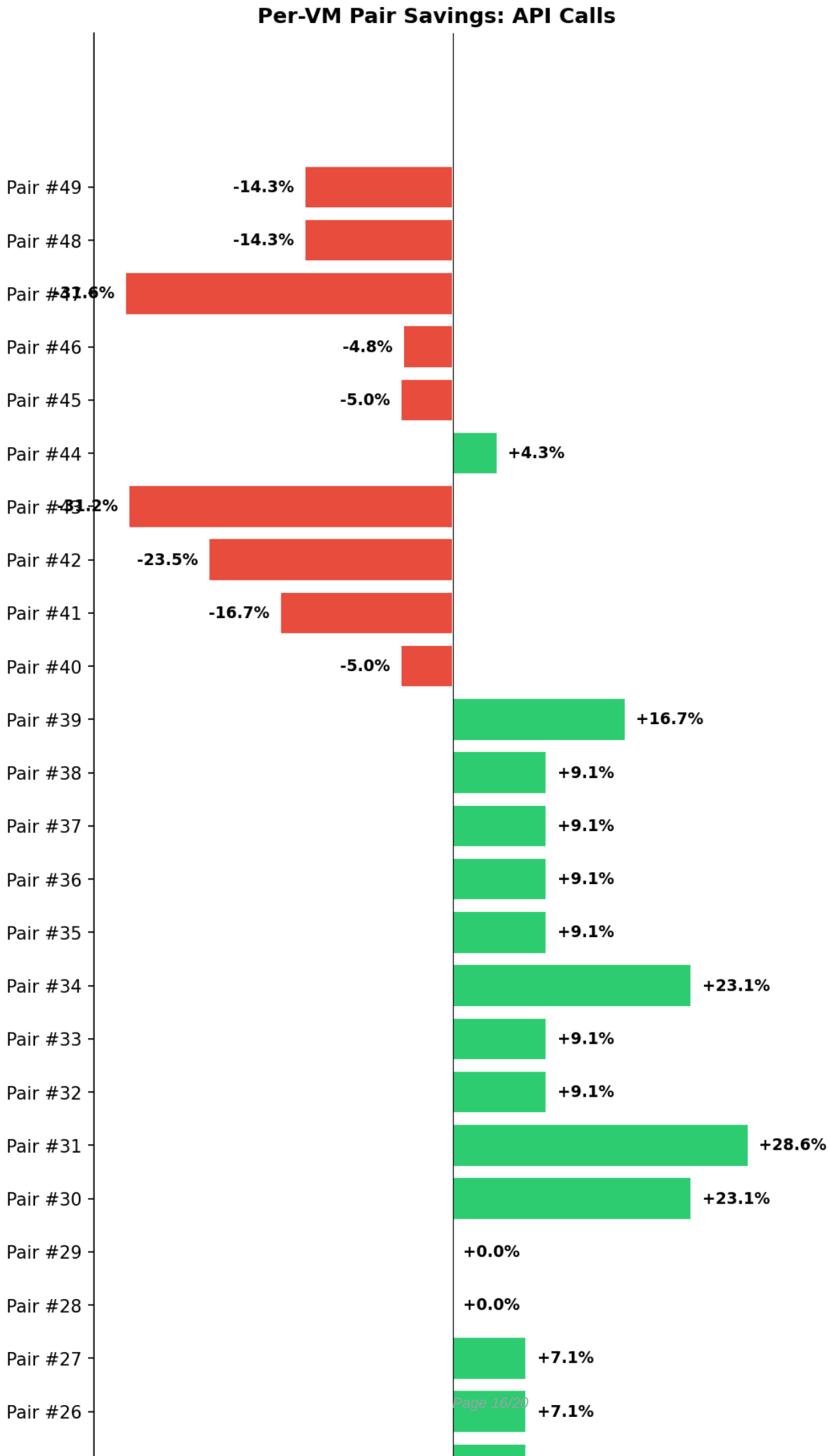
Savings % for each ON/OFF VM pair. Consistent green bars indicate reliable RTK effect. Mixed colors suggest non-deterministic variance.


Per-VM Pair Savings: Bash Result Bytes
Pair #49 +31.4%
Pair #48 +34.1%
Pair #47 +34.4%
Pair #46 +35.3%
Pair #45 +24.6%
Pair #44 +34.6%
Pair #43 +44.6%
Pair #42 +43.0%
Pair #41 +43.9%
Pair #40 +34.8%
Pair #39 +79.3%
Pair #38 +77.4%
Pair #37 +77.4%
Pair #36 +79.4%
Pair #35 +78.1%
Pair #34 +77.7%
Pair #33 +77.2%
Pair #32 +78.2%
Pair #31 +78.0%
Pair #30 +77.6%
Pair #29 +4.5%
Pair #28 +3.5%
Pair #27 +34.0%
Pair #26 +34.3%

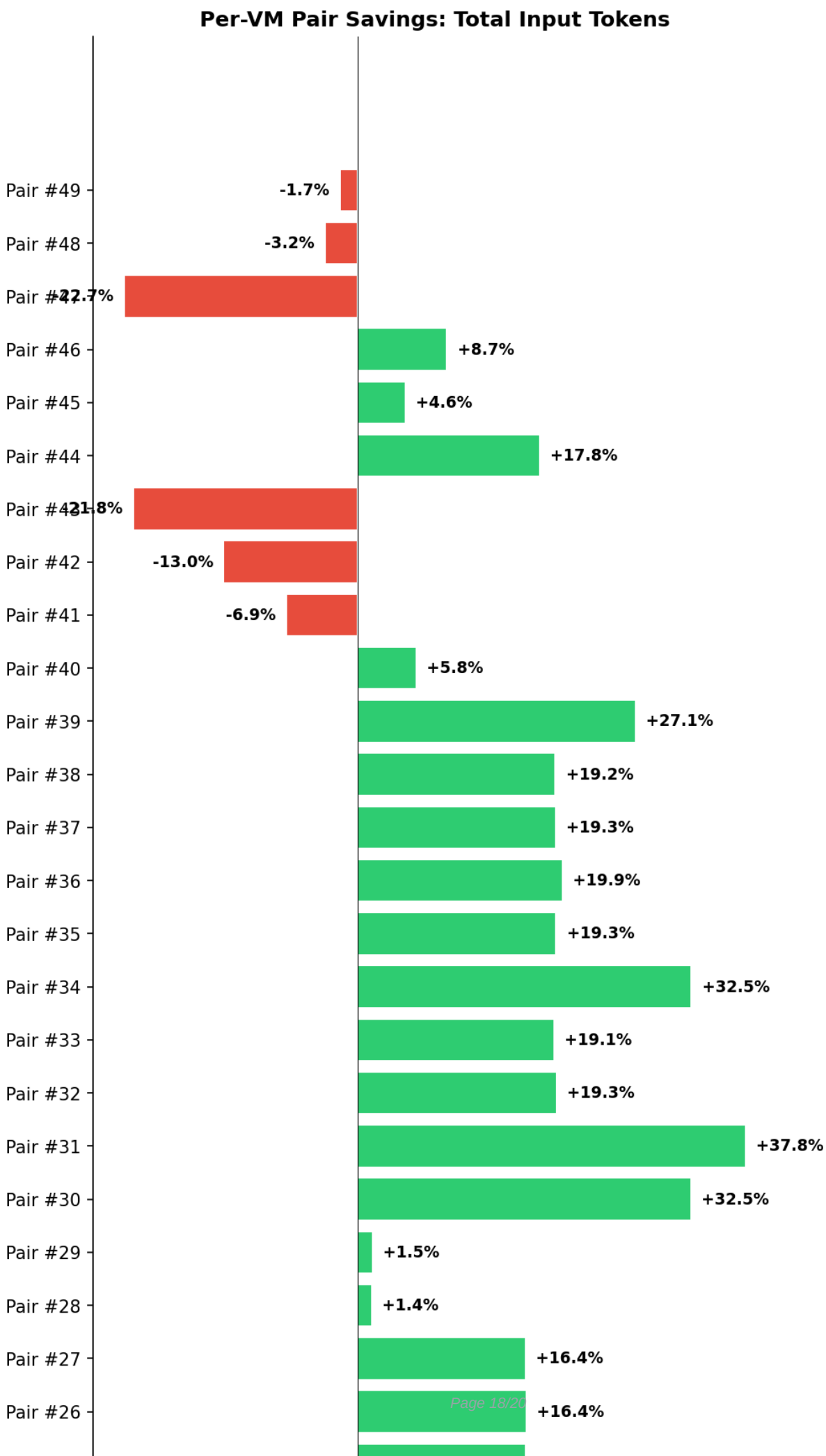
Per-VM Pair Savings (cont.)


Per-VM Pair Savings: Cost USD
Pair #49
+11.8%
Pair #48
+5.2%
Pair #47
-7.1%
Pair #46
+16.5%
Pair #45
+36.1%
Pair #44
+22.5%
Pair #43
-2.6%
Pair #42
+5.3%
Pair #41
+7.6%
Pair #40
+13.6%
Pair #39
+39.9%
Pair #38
+34.8%
Pair #37
+34.2%
Pair #36
+34.6%
Pair #35
+33.0%
Pair #34
+41.6%
Pair #33
+33.7%
Pair #32
+26.6%
Pair #31
+43.4%
Pair #30
+40.1%
Pair #29
+10.1%
Pair #28
+6.3%
Pair #27
+21.8%
Pair #26
+23.7%

Per-VM Pair Savings (cont.)


Per-VM Pair Savings: API Calls
Pair #49
-14.3%
Pair #48
-14.3%
Pair #47
-31.6%
Pair #46
-4.8%
Pair #45
-5.0%
Pair #44
+4.3%
Pair #43
-31.2%
Pair #42
-23.5%
Pair #41
-16.7%
Pair #40
-5.0%
Pair #39
+16.7%
Pair #38
+9.1%
Pair #37
+9.1%
Pair #36
+9.1%
Pair #35
+9.1%
Pair #34
+23.1%
Pair #33
+9.1%
Pair #32
+9.1%
Pair #31
+28.6%
Pair #30
+23.1%
Pair #29
+0.0%
Pair #28
+0.0%
Pair #27
+7.1%
Pair #26
+7.1%

Per-VM Pair Savings (cont.)


Per-VM Pair Savings: Total Input Tokens
Pair #49
-1.7%
Pair #48
-3.2%
Pair #47
-22.7%
Pair #46
+8.7%
Pair #45
+4.6%
Pair #44
+17.8%
Pair #43
-21.8%
Pair #42
-13.0%
Pair #41
-6.9%
Pair #40
+5.8%
Pair #39
+27.1%
Pair #38
+19.2%
Pair #37
+19.3%
Pair #36
+19.9%
Pair #35
+19.3%
Pair #34
+32.5%
Pair #33
+19.1%
Pair #32
+19.3%
Pair #31
+37.8%
Pair #30
+32.5%
Pair #29
+1.5%
Pair #28
+1.4%
Pair #27
+16.4%
Pair #26
+16.4%

Per-VM Pair Savings (cont.)

Consistency Assessment

Mixed results: 48/50 pairs show savings. Spread: 50.5pp -- high variance suggests non-determinism.

Tool Usage Analysis

Tool	Total (ON)	Avg/Sess (ON)	Total (OFF)	Avg/Sess (OFF)
Bash	679	13.6	680	13.6
Glob	4	0.1	2	0.0